

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

A Mobile Application for Bird Species Identification Using Audio Recognition and Machine Learning

By
Darragh Rohan

April 24, 2026

Minor Dissertation

**Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.**

Contents

1	Introduction	2
1.1	Context and Motivation	2
1.2	Project Objectives	3
1.3	Dissertation Structure	4
1.4	Project Repository	5
2	Methodology	6
2.1	Overview	6
2.2	Software Development Methodology	6
2.2.1	Agile and Iterative Incremental Development	6
2.2.2	Development Phases	7
2.2.3	Requirements Determination	9
2.2.4	Version Control and Collaboration	9
2.3	Testing Methodology	10
2.3.1	Test Driven Quality Assurance	10
2.3.2	Test Categories	10
2.3.3	Test Fixtures and Configuration	11
2.3.4	Continuous Integration and Delivery	11
2.4	Research Methodology	12
3	Technology Review	13
3.1	Overview	13
3.2	Frontend: React Native and Expo	13
3.2.1	Cross-Platform Mobile Development	13
3.2.2	React Native	14
3.2.3	Expo	14
3.2.4	Alternatives Considered	15
3.3	Backend: Python and FastAPI	15
3.3.1	Python	15
3.3.2	FastAPI	15
3.3.3	SQLAlchemy ORM	16
3.3.4	Alternatives Considered	16
3.4	Machine Learning: BirdNET	16
3.4.1	Audio Classification and Deep Learning	16
3.4.2	BirdNET	17
3.4.3	Audio Preprocessing: pydub and librosa	17
3.4.4	Alternatives Considered	18
3.5	Data Persistence: PostgreSQL	18
3.5.1	Relational Database Management Systems	18
3.5.2	PostgreSQL	18
3.5.3	Alternatives Considered	19

3.6.1	Object Storage	19
3.6.2	Amazon S3	19
3.6.3	Alternatives Considered	20
3.7	Authentication: Firebase Authentication	20
3.7.1	Authentication in Web and Mobile Systems	20
3.7.2	Firebase Authentication	21
3.7.3	Alternatives Considered	21
3.8	Deployment: Railway	21
3.8.1	Platform as a Service	21
3.8.2	Railway	22
3.8.3	Alternatives Considered	22
3.9	Development Tooling	22
3.9.1	Testing: pytest	22
3.9.2	CI/CD: GitHub Actions	22
3.9.3	API Documentation: Swagger UI	23
3.9.4	Database Management: pgAdmin	23
3.10	Summary	23
4	System Design	24
4.1	Overview	24
4.2	Architectural Overview	24
4.3	Frontend Design	25
4.3.1	Screen Structure and Navigation	25
4.3.2	Key Frontend Libraries	26
4.3.3	User Interface Design	27
4.4	Backend Design	27
4.4.1	Application Structure	27
4.4.2	API Endpoint Design	28
4.5	Audio Processing and ML Inference Pipeline	30
4.6	Database Design	31
4.6.1	Schema	31
4.6.2	Design Decisions	31
4.7	Security Architecture	33
4.8	Deployment Architecture	34
4.9	User Interface Screenshots	35
4.10	Summary	39
5	System Evaluation	40
5.1	Overview	40
5.2	Objective Evaluation	40
5.2.1	Objective 1: Cross-Platform Mobile Application	40

5.2.2	Objective 2: Audio Recording	41
5.2.3	Objective 3: BirdNET ML Integration	41
5.2.4	Objective 4: RESTful Backend API	42
5.2.5	Objective 5: Cloud Object Storage	42
5.2.6	Objective 6: Relational Database Persistence	43
5.2.7	Objective 7: User Authentication	43
5.2.8	Objective 8: Recording History and Playback	44
5.2.9	Objective 9: Bird Species Information	44
5.2.10	Objective 10: Automated Testing and CI/CD	44
5.2.11	Objective 11: Deployment	45
5.3	Test Results Summary	46
5.4	Key Technical Challenges and Solutions	46
5.4.1	BirdNET M4A Incompatibility	46
5.4.2	Firebase Credentials in a Cloud Environment	47
5.4.3	Secure Audio File Access with Private S3 Buckets	47
5.4.4	User Data Isolation	47
5.5	Critical Evaluation	48
5.5.1	Strengths	48
5.5.2	Limitations	48
5.6	Future Work	49
5.7	Summary	50
6	Conclusion	51
6.1	Overview	51
6.2	Project Summary	51
6.3	Evaluation of Objectives	52
6.4	Technical Achievements	53
6.5	Lessons Learned	54
6.6	Future Work	54
6.7	Closing Reflection	55
A	Project Repository and Installation Guide	56
A.1	GitHub Repository	56
A.2	Installation Instructions	56
A.2.1	Prerequisites	56
A.2.2	Running the Application	56
A.3	Deployment	57
A.3.1	Backend API	57

List of Figures

4.1	BirdID Three-Tier System Architecture.	25
4.2	BirdID Mobile Application Screen Flow.	27
4.3	BirdID Audio Processing and ML Inference Pipeline.	31
4.4	BirdID Entity-Relationship Diagram.	33
4.5	BirdID Deployment Architecture.	35
4.6	BirdID Recording Screen. Active recording state showing live duration counter and stop button.	36
4.7	Analysing screen showing a loading spinner along with a spinning leaf symbol	37
4.8	BirdID Upload Screen following a successful recording upload, displaying the identified bird.	38

List of Tables

2.1	BirdID Development Phases	8
4.1	BirdID Application Screens and Responsibilities	26
4.2	BirdID Backend Module Structure	28
4.3	BirdID RESTful API Endpoints	29
4.4	BirdID recordings Table Schema	32
5.1	BirdNET Identification Performance on Common Irish Bird Species	42
5.2	BirdID Automated Test Suite Summary	45
5.3	BirdID System Evaluation Metrics Summary	46

Chapter 1

Introduction

1.1 Context and Motivation

Bird watching and wildlife conservation have become increasingly important disciplines in the broader effort to understand and protect global biodiversity. The ability to identify bird species accurately, particularly through their vocalisations, is a skill that has traditionally required years of study and field experience. While visual identification tools have existed for decades in both print and digital formats, the challenge of automated, audio-based bird species identification presents a rich and technically demanding problem domain that sits at the intersection of machine learning, mobile computing, and cloud-based systems engineering.

Bird song is one of the most reliable indicators of species presence in a given habitat. Unlike visual sightings, which are often brief, obstructed, or dependent on weather conditions, a bird's song can be recorded and analysed with relative consistency. However, the acoustic characteristics of bird vocalisations are highly complex, varying not only between species but also by season, geographic region, and individual bird [1]. This complexity makes manual identification challenging for all but the most experienced ornithologists and creates a compelling case for the application of machine learning to the problem.

Existing applications such as Merlin Bird ID, developed by the Cornell Lab of Ornithology, have demonstrated that mobile audio identification is both technically feasible and practically valuable [2]. However, there remains an opportunity to develop a fully integrated, cloud-backed system that combines a pre-trained deep learning model with a standards-based RESTful API, scalable cloud storage, and a polished cross-platform mobile front-end. This project, *BirdID*, proposes and implements exactly such a system.

BirdID is a cross-platform mobile application that allows users to record bird audio directly on their device and receive an automated species identification pow-

ered by the BirdNET deep learning model [2], developed by the Cornell Lab of Ornithology and Chemnitz University of Technology. The system is built on a three-tier architecture: a React Native mobile frontend, a Python FastAPI backend, and a cloud data layer comprising Firebase Authentication, a PostgreSQL relational database, and AWS S3 object storage. The application supports user registration and login, audio recording and upload, real-time ML inference, recording history with playback and deletion, and an automatically fetched bird image from the iNaturalist API for each identification result.

The motivation for this project is both technical and personal. From a technical perspective, it provides an opportunity to integrate a diverse set of modern technologies and demonstrate the practical application of machine learning in a mobile context. From a personal perspective, a genuine interest in Irish wildlife and ornithology lends the application practical utility for fellow bird watchers, educators, and conservation groups.

1.2 Project Objectives

The following objectives were defined at the outset of the project and constitute the measurable criteria against which the success of the system is evaluated in Chapter 5.

1. **Cross-Platform Mobile Application:** Develop a cross-platform mobile application using React Native and Expo, supporting both iOS and Android, with a professional Merlin-inspired interface featuring bottom tab navigation and a smooth multi-screen user flow.
2. **Audio Recording:** Implement high-quality-in-app-audio recording at 44.1kHz using the `expo-av` library, with real-time duration tracking, waveform animation, and microphone permission handling.
3. **BirdNET ML Integration:** Integrate the BirdNET deep learning model via the `birdnetlib` Python library to perform automated bird species identification from recorded audio, returning a species common name, scientific name, confidence score, and a list of alternative detections.
4. **RESTful Backend API:** Build a Python FastAPI backend server to handle audio file receipt, M4A-to-WAV format conversion, machine learning inference, and data persistence, exposing eight documented endpoints communicating via HTTPS with JWT-authenticated requests.

5. **Cloud Object Storage:** Implement scalable audio file storage using AWS S3, with a private bucket, presigned URL generation for secure temporary access, and automatic cleanup of temporary local files following upload.
6. **Relational Database Persistence:** Store structured recording metadata, including species identification results, confidence scores, S3 file paths, timestamps, and user identifiers, in a PostgreSQL relational database using the SQLAlchemy ORM.
7. **User Authentication:** Implement secure user registration and login using Firebase Authentication, with JWT token verification middleware on all protected backend endpoints and server-side user data isolation.
8. **Recording History and Playback:** Allow authenticated users to retrieve a personalised, chronologically sorted history of their recordings, with the ability to stream audio playback directly from S3 via presigned URLs and delete individual entries.
9. **Bird Species Information:** Automatically fetch and display a representative bird image for each identification result using the iNaturalist API, with graceful fallback handling for species with no available image.
10. **Automated Testing and CI/CD:** Implement a comprehensive suite of automated tests using pytest across eight test files, covering API endpoints, database models, authentication, S3 integration, and BirdNET functionality, executed automatically on every push to the main branch via GitHub Actions with coverage reporting via Codecov.
11. **Deployment:** Deploy the containerised backend and managed PostgreSQL database to the Railway cloud platform, with automatic deployments triggered by Git push, environment variable management, and HTTPS/SSL configured automatically.
12. **Mobile Application Distribution:** Package and distribute the Android application as an installable APK using Expo Application Services (EAS), enabling end users to install and run the application on physical devices without requiring a development environment, supported by QR code and direct download access.

1.3 Dissertation Structure

The remainder of this dissertation is structured as follows.

Chapter 2 describes the **Methodology** adopted throughout the project, covering both the software development methodology and the research approach, including the use of an Agile, iterative and incremental development process, the role of GitHub as a version control and CI/CD platform, and the testing strategy employed.

Chapter 3 presents the **Technology Review**, a comprehensive survey of the key technologies, frameworks, and standards employed in the system. Each technology is examined at a conceptual level, with academic and technical justification for its selection and discussion of the alternatives considered.

Chapter 4 details the **System Design**, providing a thorough description of the three-tier system architecture, the audio processing and ML inference pipeline, the database schema, the RESTful API design, the security architecture, and the interactions between all system components, illustrated with architectural and data flow diagrams.

Chapter 5 constitutes the **System Evaluation**, in which the completed system is assessed against each of the objectives defined above. The automated test results are presented, the performance of BirdNET on Irish bird species is discussed, and the strengths and limitations of the system are critically examined.

Chapter 6 provides the **Conclusion**, summarising the project outcomes, reflecting on lessons learned, identifying limitations, discussing the proposed custom algorithm as future work, and suggesting further directions for development.

1.4 Project Repository

The full source code for this project, including the React Native mobile application, the Python FastAPI backend, all test files, the GitHub Actions CI/CD configuration, and installation instructions, is available at the following URL:

<https://github.com/darragh50/BirdID>

The repository is organised into two primary directories: `/frontend`, containing the React Native Expo application across eight screens and fifteen or more components, and `/backend`, containing the FastAPI server, database models, BirdNET integration, S3 and Firebase configuration modules, and the full pytest test suite across eight test files. A `README.md` file at the root of the repository provides full setup and installation instructions.

Chapter 2

Methodology

2.1 Overview

This chapter describes the approach taken to both the software development and the research dimensions of the BirdID project. A clearly defined methodology is essential in any software engineering project of this scale, as it provides a framework for managing complexity, tracking progress, and ensuring that each phase of development builds meaningfully upon the last. The approach adopted for this project draws on Agile principles, applying an iterative and incremental development model throughout the six-month development period from October 2025 to April 2026.

2.2 Software Development Methodology

2.2.1 Agile and Iterative Incremental Development

The software development methodology adopted for BirdID was an iterative and incremental approach, informed by Agile principles [3]. Rather than attempting to design and implement the entire system upfront, the project was divided into a series of small, functional phases, each delivering a working increment of software that could be tested, evaluated, and built upon in subsequent phases. This approach is well suited to projects where requirements may evolve over time and where early feedback is valuable in shaping later design decisions.

The Agile methodology is widely adopted in the software industry precisely because it accommodates change and promotes continuous delivery of working software [4]. In the context of this project, the architecture enabled the independent development, integration, and verification of each major component: audio recording, cloud storage, authentication, and machine learning integration. By

ensuring each module was functional before introducing the next layer of complexity, this incremental approach prevented the accumulation of unresolved technical debt and maintained a working prototype at every stage.

2.2.2 Development Phases

The project was structured into eleven phases over the course of the academic year, each targeting a specific functional area of the system. Table 2.1 summarises these phases, their focus areas, and their approximate durations.

Each phase was considered complete only when its core deliverable was functional and integrated with the existing system. For example, the cloud storage phase (Phase 4) was not closed until audio files were being successfully uploaded to S3, their URLs persisted in PostgreSQL, and temporary local files cleaned up automatically.

#	Phase	Key Activities	Duration
1	Foundation & Setup	Environment setup, project scaffolding, Git repository initialisation, basic React Native and FastAPI skeletons	Weeks 1–2
2	Core Audio	Audio recording via expo-av, microphone permissions, file upload to backend	Weeks 3–4
3	Database Integration	PostgreSQL setup, SQLAlchemy ORM, schema design, metadata persistence	Weeks 5–6
4	Cloud Storage	AWS S3 integration, Boto3, presigned URL generation, local file cleanup	Weeks 7–8
5	Authentication	Firebase Auth (frontend SDK and backend Admin SDK), JWT middleware, login/signup UI	Weeks 9–10
6	ML Integration	BirdNET library setup, M4A to WAV conversion, inference pipeline, result parsing	Weeks 11–12
7	Playback & History	Audio streaming from S3, recording list UI, delete functionality	Weeks 13–14
8	UI/UX Overhaul	implementation of bottom tab navigation, waveform animation, iNaturalist API integration	Weeks 15–17
9	Testing & QA	pytest suite, CI/CD pipeline, manual device testing	Week 18
10	Deployment	Railway platform setup, PostgreSQL migration, environment configuration, HTTPS, and mobile application build and distribution via Expo Application Services (EAS)	Week 19
11	Documentation	Architecture diagrams, dissertation writing, user guide	Weeks 20–22

Table 2.1: BirdID Development Phases

2.2.3 Requirements Determination

Requirements for the project were determined through a combination of personal domain knowledge, examination of existing bird identification applications (particularly Merlin Bird ID), and an initial project proposal agreed with the project supervisor in October 2025. The proposal defined the core problem domain, the high-level objectives, the proposed technology stack, and a schedule of work. This document served as the primary planning artefact throughout Semester 1.

As the project progressed, requirements were refined iteratively. For instance, the decision to implement a dedicated Analysing screen with a progress indicator emerged from informal user experience evaluation during the UI/UX overhaul phase, rather than being specified at the outset. This flexibility is a key benefit of the Agile approach and reflects the reality of software development in practice.

2.2.4 Version Control and Collaboration

Git and GitHub were used throughout the project for version control, progress tracking, and continuous integration. The repository at <https://github.com/darragh50/BirdID> accumulated over one hundred and fifty commits across the development period, with a commit frequency of approximately six to seven commits per week during active development phases. Descriptive commit messages were used consistently, providing a clear audit trail of the development process. Examples of commit messages include:

- Add PostgreSQL dependencies (psycopg2 and SQLAlchemy)
- Add and initialise S3 client
- Add firebase web SDK and Asyncstorage for authentication
- Update App.js with navigation and auth based routing
- Add deployment config files for Railway

The repository is organised into clearly separated directories: `/frontend` for the React Native application, `/backend` for the FastAPI server and all associated modules, `/migrations` for database migration scripts, and `/tests` for the full pytest suite. This structure reflects industry-standard practices for a full-stack project organisation.

2.3 Testing Methodology

2.3.1 Test Driven Quality Assurance

Testing was treated as a top concern throughout the development process. Rather than deferring all testing to a single phase at the end of development, tests were written incrementally alongside the corresponding features, and the full suite was executed automatically on every push to the main branch via GitHub Actions. This approach ensures that regressions are detected immediately and that the codebase remains in a consistently functional state.

The testing framework used for the backend was `pytest` [5], selected for its clean syntax, powerful fixture system, and excellent integration with FastAPI's `TestClient`. Eight test files were produced across four test categories, as described below.

2.3.2 Test Categories

Unit Tests (`@pytest.mark.unit`) test individual functions and classes in isolation. The database model tests in `test_models.py` fall into this category, verifying that `Recording` model instances can be created and persisted correctly, that optional fields default to `None` as expected, and that the `__repr__` method returns a correctly formatted string representation.

API Tests (`@pytest.mark.api`) test the HTTP endpoints of the FastAPI backend using FastAPI's built in `TestClient`, which allows requests to be simulated without launching a live server. These tests verify that all protected endpoints correctly reject unauthenticated requests with a 401 or 403 status code, that the root and health check endpoints return the expected response structures, and that the database and S3 health endpoints confirm connectivity to their respective services.

Integration Tests (`@pytest.mark.integration`) test the interaction between multiple system components. The BirdNET tests in `testbirdnet.py` verify that the model is correctly configured, that the `identify_bird_from_file` function handles missing files gracefully, that the `get_best_match` function correctly returns `None` for an empty detection list, and that the `format_bird_result` function produces correctly formatted output including the confidence percentage. The database integration tests in `testdb.py` verify end to end database connectivity, record insertion, and cleanup.

Slow Tests (`@pytest.mark.slow`) cover operations that involve real external service calls, such as the S3 upload test in `tests3.py`, which creates a sample file, uploads it to the live S3 bucket, and verifies a successful response. These tests are marked separately so they can be excluded from rapid local test runs when

required.

2.3.3 Test Fixtures and Configuration

Shared test infrastructure is defined in `conftest.py`, which provides three reusable pytest fixtures. The `client` fixture creates a fresh FastAPI `TestClient` for each test function, ensuring test isolation. The `testdb` fixture creates all database tables, opens a new SQLAlchemy session, yields it to the test, and closes it upon completion. The `mock_firebase_token` fixture provides a mock JWT token string for use in authentication related tests, avoiding the need to call the live Firebase service during unit and API testing.

2.3.4 Continuous Integration and Delivery

The CI/CD pipeline was implemented using GitHub Actions and is defined in the repository's workflow configuration file. On every push to the main branch, the pipeline performs the following steps automatically:

1. Spin up an Ubuntu Linux runner environment.
2. Launch a PostgreSQL 15 service container with a test database.
3. Install all Python dependencies from `requirements.txt`.
4. Set environment variables, including the test database URL and Firebase credentials decoded from a Base64-encoded repository secret.
5. Execute the full pytest suite with verbose output.
6. Re-execute pytest with `pytest-cov` to generate an XML coverage report.
7. Upload the coverage report to Codecov for tracking over time.
8. Install Node.js 18 and verify that frontend dependencies install without errors.

The pipeline is configured to fail the build if any test fails, providing immediate feedback and preventing broken code from remaining undetected in the main branch. The use of GitHub Secrets to manage AWS credentials, Firebase credentials, and the database URL ensures that no sensitive information is ever committed to the repository.

2.4 Research Methodology

The research component of this project involved identifying, evaluating, and selecting appropriate technologies for each layer of the system. This process was crucial, as technologies were not selected on the basis of familiarity, but were evaluated against defined criteria including performance, compatibility with the Python and JavaScript ecosystems, documentation quality, community support, cost, and suitability for a student development environment.

For each major technology decision, including framework selection, database choice, cloud storage provider, authentication service, and ML model, alternatives were identified and assessed. For example, MongoDB was considered as an alternative to PostgreSQL and rejected on the grounds that a relational structure is better suited to the user recording relationship present in this system. Node.js with Express was considered as an alternative to FastAPI and rejected because Python's machine learning and audio processing ecosystem is considerably more mature, supported by a rich scientific computing stack including libraries such as NumPy and SciPy [6]. These decisions are documented and justified in detail in Chapter 3.

Background research was conducted using a combination of official technical documentation, academic papers (particularly regarding the BirdNET model and audio feature extraction), and reputable practitioner resources. All sources are cited in IEEE format and collated in the references section.

Chapter 3

Technology Review

3.1 Overview

This chapter provides a comprehensive review of the technologies, frameworks, libraries, and services employed in the development of BirdID. For each technology, the underlying concepts are examined, the rationale for selection is justified, and the principal alternatives considered are discussed. The technologies are organised by system layer, including frontend, backend, machine learning, data persistence, cloud storage, authentication, deployment, and development tooling. This review demonstrates that each technology choice was made deliberately, informed by research, and appropriate to the requirements of the system.

3.2 Frontend: React Native and Expo

3.2.1 Cross-Platform Mobile Development

Mobile application development has historically required separate codebases for iOS and Android, each written in the platform’s native language: Swift for iOS, and Java or Kotlin for Android [7]. This approach, while delivering optimal performance and full access to platform APIs, imposes a significant development overhead, requiring duplicate implementations of every feature for two distinct ecosystems.

Cross-platform frameworks address this problem by allowing developers to write a single codebase that compiles or runs on both platforms. The most prominent frameworks in this space are React Native, developed by Meta, and Flutter, developed by Google [8]. Both have matured significantly in recent years and are widely adopted in industry.

3.2.2 React Native

React Native [9] is an open-source framework that allows mobile applications to be built using JavaScript and the React programming model. Unlike web-based hybrid frameworks such as Ionic, which render application content inside a `WebView`, React Native renders genuine native UI components on both iOS and Android. This means that a React Native button, for instance, renders as a native `UIButton` on iOS and a native `Button` on Android, providing a user experience that is indistinguishable from a natively developed application [10].

React Native was selected for BirdID for several reasons. First, it supports both iOS and Android from a single JavaScript codebase, reducing development time significantly. Second, its component-based architecture, inherited from React, promotes modular and maintainable code. Third, its large community and extensive ecosystem of third-party libraries provide ready-made solutions for common mobile development challenges, including audio recording, navigation, and storage.

3.2.3 Expo

Expo [11] is a platform and set of tools built on top of React Native that further simplifies the development process. The Expo managed workflow removes the need to configure native build tools such as Xcode or Android Studio during development, instead providing a unified development server and a companion application, Expo Go, through which the application can be run on a physical device by scanning a QR code. It also supports application packaging and distribution via Expo Application Services (EAS), enabling the generation of installable builds for real devices. This significantly accelerates the development and testing cycle.

Of particular relevance to BirdID is the `expo-av` library, which provides a consistent, cross-platform API for audio recording and playback. Audio recording on mobile platforms involves handling microphone permissions, managing audio sessions, and producing correctly formatted output files. These tasks differ substantially between iOS and Android at the native level. The `expo-av` library abstracts these differences, producing M4A (AAC-encoded) audio files on both platforms with a uniform programming interface [12].

Additional Expo and React Native libraries used in BirdID include `@react-navigation/native` and `@react-navigation/bottom-tabs` for screen navigation, `expo-linear-gradient` for visual UI polish, `@expo/vector-icons` (Ionicons) for iconography, and `@react-native-async-storage` for local persistence of the Firebase authentication token between sessions.

3.2.4 Alternatives Considered

Flutter was the primary alternative considered. Flutter uses the Dart programming language and renders UI components using its own graphics engine rather than native platform components, which provides highly consistent visual output across platforms [8]. However, Flutter was rejected for BirdID as the development process was better aligned with the JavaScript ecosystem, with which the developer has greater experience, and the React Native community provides more mature tooling for audio processing on mobile.

3.3 Backend: Python and FastAPI

3.3.1 Python

Python 3.11 was selected as the primary backend language for BirdID. Python is the dominant language in the machine learning and data science ecosystem, and its selection was largely driven by the requirement to integrate the BirdNET bird identification model, which is native to Python [13]. Beyond machine learning, Python's extensive standard library and the maturity of its audio processing ecosystem, particularly the `pydub` and `librosa` libraries, made it the natural choice for a system that must perform audio format conversion and signal processing as part of its core pipeline.

3.3.2 FastAPI

FastAPI [14] is a modern, high-performance Python web framework for building APIs, built on top of the ASGI (Asynchronous Server Gateway Interface) standard. It was released in 2018 and has seen rapid adoption in the Python community, consistently ranking among the most popular Python web frameworks in developer surveys [15].

FastAPI offers several features that made it particularly well-suited to BirdID. First, it provides automatic interactive API documentation via Swagger UI at the `/docs` endpoint, generated from the application's type annotations without any additional configuration. This was invaluable during development for testing endpoints manually. Second, it uses Pydantic for request and response validation, meaning that type errors and malformed inputs are caught automatically at the framework level rather than requiring manual validation code. Third, its native support for asynchronous request handling allows the server to handle concurrent file uploads efficiently without blocking.

FastAPI applications are served using Uvicorn [16], an ASGI server that provides excellent performance characteristics and is the recommended server for

FastAPI in both development and production environments.

3.3.3 SQLAlchemy ORM

SQLAlchemy [17] is the Object-Relational Mapping (ORM) library used in BirdID to interact with the PostgreSQL database. An ORM maps Python class definitions to relational database tables, allowing database operations to be expressed as Python method calls rather than raw SQL strings. This improves code readability, reduces the risk of SQL injection vulnerabilities, and makes schema changes easier to manage. In BirdID, the `Recordings` model class is defined using SQLAlchemy's declarative base system, with each column defined as a class attribute with an explicit type and set of constraints.

3.3.4 Alternatives Considered

Node.js with Express was the principal alternative considered for the backend. Express is a mature, widely used framework with a large ecosystem and strong community support [18]. It was rejected primarily because Python's machine learning and audio processing libraries are considerably more mature and better documented than their JavaScript equivalents. Since BirdNET is a Python library, using Node.js would have required either a separate Python microservice for ML inference or a foreign function interface, both of which would have added unnecessary architectural complexity.

3.4 Machine Learning: BirdNET

3.4.1 Audio Classification and Deep Learning

The automatic identification of bird species from audio recordings is a specialised application of the broader field of environmental sound classification. Traditional approaches to this problem have relied on the manual extraction of acoustic features, such as Mel-Frequency Cepstral Coefficients (MFCCs), spectral centroid, and zero crossing rate, followed by classification using shallow machine learning models such as Support Vector Machines or k-Nearest Neighbours [19]. While these approaches are computationally efficient and interpretable, they have been consistently outperformed by deep learning methods, particularly Convolutional Neural Networks (CNNs), which are capable of learning rich, hierarchical feature representations directly from raw audio spectrograms [20].

3.4.2 BirdNET

BirdNET [2] is an open source deep learning model developed jointly by the Cornell Lab of Ornithology and Chemnitz University of Technology, specifically designed for bird species identification from audio recordings. It is among the most capable and widely used bird identification models available, trained on a dataset covering over 6,000 bird species globally.

The model operates by first converting the input audio signal into a mel-spectrogram which is a two-dimensional time-frequency representation in which frequency is mapped to the perceptually motivated mel scale [21]. This spectrogram is then passed through a deep Convolutional Neural Network consisting of multiple convolutional layers, each learning progressively more abstract representations of the acoustic patterns present in the recording. The final layers of the network produce a probability distribution over all species in the model's vocabulary, from which the top predictions and their associated confidence scores are returned.

BirdNET is accessed in BirdID via the `birdnetlib` Python library [22], which provides a clean, high-level interface to the underlying model. The library accepts audio files in WAV format, performs the spectrogram conversion and inference internally, and returns a list of detection objects, each containing the common name, scientific name, confidence score, and the start and end time of the detected vocalisation within the recording.

BirdNET was selected for BirdID over training a custom model for several reasons. Training a bird identification model from scratch would require a large labelled dataset, substantial computational resources, and extensive experimentation, all of which are beyond the scope of a single-student project [2]. BirdNET, by contrast, is freely available under a Creative Commons licence (CC BY-NC-SA 4.0), supports geographic filtering to improve accuracy for regional species, and has been validated in published research to achieve accuracy in the range of 85–90% for common species.

3.4.3 Audio Preprocessing: pydub and librosa

A key technical challenge in integrating BirdNET with the BirdID system was the compatibility of the audio format. The `expo-av` library on the mobile frontend records audio in M4A format (AAC codec), which is the default format on both iOS and Android. BirdNET's underlying `librosa` dependency has limited support for M4A files, making direct analysis unreliable.

This was resolved by implementing an automatic conversion step in the backend pipeline using `pydub` [23], a Python library for audio manipulation that uses FFmpeg as its backend. When a recording is received by the backend, it is converted

from M4A to WAV at 44.1kHz mono before being passed to BirdNET for analysis. The WAV file is used solely for analysis and is deleted immediately afterwards, while the original M4A file is preserved and uploaded to AWS S3 for storage. The conversion adds approximately one to two seconds to the total processing time and is entirely transparent to the user.

`librosa` [24] is a Python library for audio and music signal analysis, widely used in the research community. In BirdID it is used indirectly as a dependency of `birdnetlib`, performing the low-level audio loading and feature extraction operations that underpin BirdNET’s inference pipeline.

3.4.4 Alternatives Considered

Google’s Chirp model was identified as a potential alternative to BirdNET. Chirp is a large-scale bird sound classification model developed by Google Research, reported to achieve high accuracy across a wide range of species [25]. However, it was not publicly available via an accessible API at the time of development and was therefore not a viable option. The Merlin Sound ID system, developed by the Cornell Lab, was also considered, but it does not expose a public API and operates as a closed ecosystem within the Merlin Bird ID application. Training a custom model was considered but rejected as described above.

3.5 Data Persistence: PostgreSQL

3.5.1 Relational Database Management Systems

A relational database management system (RDBMS) stores data in structured tables with defined schemas, enforcing data integrity through constraints, primary and foreign keys, and the ACID (Atomicity, Consistency, Isolation, Durability) properties [26]. These properties ensure that database transactions are processed reliably, making RDBMSs the appropriate choice for systems in which data correctness and consistency are paramount.

3.5.2 PostgreSQL

PostgreSQL [27] is a powerful, open-source RDBMS with over thirty years of active development. It is widely regarded as one of the most reliable and feature-rich database systems available, supporting advanced data types, full-text search, JSON storage, and a rich set of indexing options. It is ACID-compliant, making it suitable for production use cases where data integrity is critical.

In BirdID, PostgreSQL stores all structured recording metadata in a single `recordings` table. The schema is normalised to Third Normal Form (3NF), with the `user_id` column indexed to support efficient user-specific queries. Timezone-aware timestamps are used throughout, and nullable fields accommodate optional data such as GPS coordinates and bird identification results. PostgreSQL was hosted locally during development using pgAdmin 4 as a management interface, and subsequently migrated to a managed Railway instance for production deployment.

3.5.3 Alternatives Considered

MongoDB, a document-oriented NoSQL database, was considered as an alternative. MongoDB stores data as flexible JSON-like documents, which can be advantageous for rapidly evolving schemas [28]. It was rejected for BirdID on the grounds that the relationship between users and recordings is inherently relational and is better represented in a tabular schema with enforced types and constraints. SQLite was also considered for its simplicity and zero-configuration setup but was rejected because it does not support concurrent write operations, making it unsuitable for a production server environment.

3.6 Cloud Object Storage: AWS S3

3.6.1 Object Storage

Object storage is a data storage architecture in which data is stored as discrete objects, each comprising the data itself, metadata, and a unique identifier, rather than as files in a hierarchical directory structure [29]. This architecture is particularly well suited to storing large binary files such as audio recordings, as it decouples storage from the application server and enables direct client access to stored objects via URLs.

3.6.2 Amazon S3

Amazon Simple Storage Service (S3) [30] is the industry-standard object storage service provided by Amazon Web Services. It offers virtually unlimited storage capacity, a durability guarantee of 99.99999999% (eleven nines), and a rich set of access control features. S3 is widely used in industry for storing audio, video, images, and other binary data that would be inappropriate to store directly in a relational database.

In BirdID, S3 is used to store all audio recordings uploaded by users. The bucket is configured as private, with no public access permitted. Access to stored files is granted through presigned URLs, which are time-limited, cryptographically signed URLs generated by the backend using the `boto3` AWS SDK [31]. Presigned URLs in BirdID expire after one hour, after which they become invalid and a new URL must be generated. This approach ensures that audio files are never permanently exposed, while still allowing the mobile frontend to stream recordings directly from S3 without routing audio data through the backend server, which would introduce an unnecessary performance bottleneck.

The S3 bucket is located in the `eu-north-1` region (Stockholm), selected as the geographically closest AWS region to Ireland, minimising latency for upload and retrieval operations.

3.6.3 Alternatives Considered

Firebase Storage, Google Cloud Storage, and local filesystem storage were considered as alternatives. Firebase Storage was rejected on the grounds that AWS S3 offers more flexible pricing, better documentation, and greater industry adoption. Google Cloud Storage was rejected in favour of S3 for the same reasons, and because the rest of the cloud infrastructure was already AWS- based. Local filesystem storage was initially used during early development to validate the recording pipeline. However, this approach was not suitable for a production environment. This was because files stored on the server filesystem are lost when the server restarts, do not scale beyond a single server instance, and take up a large amount of local storage.

3.7 Authentication: Firebase Authentication

3.7.1 Authentication in Web and Mobile Systems

User authentication is a fundamental security concern in any multi-user application. A robust authentication system must handle user registration, credential validation, session management, token refresh, and password reset securely. Implementing these features from scratch is time-consuming and introduces significant risk, as any vulnerability in a custom authentication implementation can compromise the security of all user data [32].

3.7.2 Firebase Authentication

Firebase Authentication [33] is a managed authentication service provided by Google as part of the Firebase platform. It handles user registration, login, session management, and token refresh transparently, allowing developers to integrate secure authentication without implementing any of the underlying cryptographic infrastructure.

Firebase uses JSON Web Tokens (JWTs) [34] as its authentication mechanism. When a user logs in, Firebase returns a signed JWT containing the user's unique identifier (UID) and an expiry timestamp. This token is stored on the mobile device using `AsyncStorage` and included in the `Authorization` header of every subsequent API request. On the backend, the Firebase Admin SDK verifies the token's cryptographic signature, confirms that it has not expired, and extracts the user UID. Critically, the user UID is always extracted from the verified token on the server-side, never from the request body, which prevents client-side spoofing of another user's identity.

This architecture ensures complete user data isolation: every database query in BirdID is filtered by the verified `user_id`, making it impossible for one user to access or modify another user's recordings.

3.7.3 Alternatives Considered

A custom JWT authentication system using `bcrypt` password hashing was considered as an alternative. While this approach would have provided more direct control over the authentication flow, it was rejected on the grounds that it would have required approximately ten to fifteen additional development hours and introduced a significant risk of security vulnerabilities if implemented incorrectly. Firebase Authentication provides tested, enterprise-grade security out of the box and was therefore the appropriate choice for a project of this scope.

3.8 Deployment: Railway

3.8.1 Platform as a Service

Platform as a Service (PaaS) [35] is a cloud computing model in which the provider manages the underlying infrastructure including servers, networking, operating systems, and runtime environments, while the developer focuses solely on deploying and managing their application. PaaS platforms dramatically reduce the operational overhead of deploying web applications compared to Infrastructure as a Service (IaaS) solutions such as AWS EC2 or DigitalOcean Droplets, where the developer is responsible for configuring the server environment from scratch.

3.8.2 Railway

Railway [36] is a modern PaaS platform that supports Git-based deployment workflows. When a push is made to the main branch of the GitHub repository, Railway automatically detects the change, builds the application, and deploys it to a containerised cloud environment. This push-to-deploy workflow eliminates the need for manual deployment steps and ensures that the production environment always reflects the latest state of the main branch.

Railway was selected for BirdID for several reasons. It provides a managed PostgreSQL database instance with automatic backups, removing the need to configure and maintain a separate database server. It automatically provisions HTTPS/SSL certificates, ensuring that all client-server communication is encrypted. It supports environment variable management through a dashboard interface, allowing secrets such as AWS credentials and Firebase credentials to be stored securely without being committed to the repository. Its free tier provides five US dollars of monthly credit, which is sufficient for development and demonstration purposes.

3.8.3 Alternatives Considered

Heroku was the most established PaaS alternative at the time of development, but was rejected following the removal of its free tier in 2022 [37]. Render was also considered and is functionally very similar to Railway. Railway was ultimately preferred for its slightly more generous free tier and cleaner dashboard interface. AWS EC2 and DigitalOcean were considered but rejected as requiring substantially more DevOps configuration than is appropriate for a student project of this scope.

3.9 Development Tooling

3.9.1 Testing: pytest

pytest [5] is the testing framework used for all backend tests in BirdID. It was selected for its clean, readable test syntax, powerful fixture system, support for test categorisation via markers, and excellent integration with FastAPI's `TestClient`. pytest is the de facto standard testing framework for Python projects and is widely used in both industry and academia.

3.9.2 CI/CD: GitHub Actions

GitHub Actions [38] is the CI/CD platform used to automate the execution of the test suite on every push to the main branch. It was selected because it is natively

integrated with GitHub, requiring no external service configuration, and because it supports service containers. Thus allowing a live PostgreSQL instance to be spun up as part of the test pipeline. Coverage reports are uploaded to Codecov [39] automatically as part of the pipeline.

3.9.3 API Documentation: Swagger UI

FastAPI automatically generates interactive API documentation using the OpenAPI specification [40] and serves it via Swagger UI at the `/docs` endpoint. This documentation was used extensively during development for manual endpoint testing and is included as part of the project’s technical documentation.

3.9.4 Database Management: pgAdmin

pgAdmin 4 [41] was used as the graphical management interface for PostgreSQL during local development, providing a query builder, schema visualisation, and data inspection tools. Postman [42] was used alongside the Swagger UI for manual API endpoint testing during development.

3.10 Summary

This chapter has reviewed the full technology stack employed in BirdID, examining each component at a conceptual level and justifying its selection in the context of the system’s requirements. The choices made reflect a deliberate, research-informed approach to system design. Industry standard tools were preferred over bespoke solutions where appropriate, alternatives were considered and rejected on clearly articulated grounds, and each technology was evaluated for its compatibility with the other components of the stack. The following chapter builds directly on this review by describing how these technologies were integrated into the overall system architecture. Particular attention was given to ensuring compatibility and seamless integration between all selected technologies.

Chapter 4

System Design

4.1 Overview

This chapter provides a comprehensive description of the architecture and design of the BirdID system. It covers the high-level three-tier architecture, the detailed component design of each tier, the audio processing and machine learning inference pipeline, the database schema, the RESTful API design, the security architecture, and the deployment configuration. Where appropriate, diagrams are used to illustrate the relationships between components and the flow of data through the system. The design decisions described here are directly informed by the technology review presented in Chapter 3.

4.2 Architectural Overview

BirdID is designed as a three tier client-server application, a well established architectural pattern in which the system is divided into a presentation layer, an application layer, and a data layer, each with clearly defined responsibilities and interfaces [43]. This separation of concerns improves maintainability, allows each tier to be scaled independently, and makes the system easier to reason about and test.

The three tiers in BirdID are as follows. The **presentation layer** is the React Native mobile application, responsible for all user interaction, audio recording, and the display of identification results and recording history. The **application layer** is the Python FastAPI backend server, responsible for authentication verification, audio processing, machine learning inference, and coordination between the data services. The **data layer** comprises three distinct services: a PostgreSQL relational database for structured metadata, an AWS S3 bucket for binary audio file storage, and the BirdNET machine learning model for species inference. An

additional external service, Firebase Authentication is decoupled from the three tiers but supports both the frontend and the backend.

The architecture follows a loosely coupled design. The frontend and backend communicate exclusively via stateless RESTful HTTP requests, ensuring that each layer can evolve independently without tight dependencies. Within the backend, integration with external services such as PostgreSQL, AWS S3, and Firebase Authentication is handled through clearly defined service modules, promoting high cohesion within components and low coupling between them. This improves maintainability, scalability, and testability.

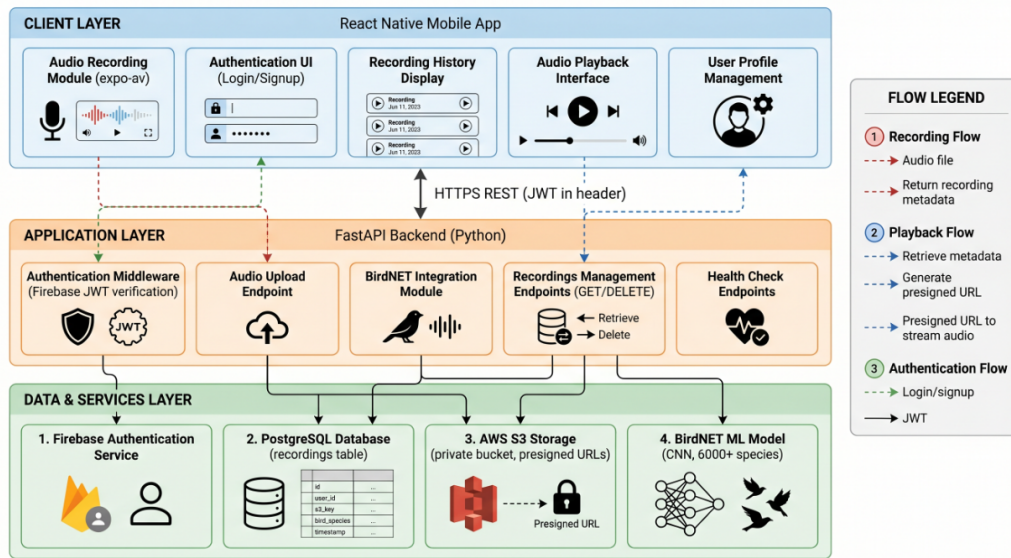


Figure 4.1: BirdID Three-Tier System Architecture.

Figure 4.1 illustrates the overall system architecture. All communication between the mobile client and the backend server takes place over HTTPS, with every request carrying a Firebase JWT token in the **Authorization** header. The backend verifies this token before processing any request, ensuring that unauthenticated access is impossible at the application layer.

4.3 Frontend Design

4.3.1 Screen Structure and Navigation

The BirdID mobile application consists of eight screens, organised into a bottom tab navigation structure inspired by the Merlin Bird ID application. The bottom

tab navigator provides three primary tabs: **Home**, **My Birds**, and **Profile**. From the Home tab, users navigate to three additional screens that form the core recording flow: the Recording screen, the Analysing screen, and the Results screen. Two further screens, Login and Signup, are presented before the main tab navigator when no authenticated session exists. Table 4.1 summarises the eight screens and their responsibilities.

Screen	Responsibility
Login	Email and password login via Firebase Authentication.
Signup	New user registration via Firebase Authentication.
Home (Tab 1)	Landing screen with prominent record button and navigation to the recording flow.
Recording	Audio recording interface with live waveform animation, duration timer, and start/stop controls.
Analysing	Progress indicator screen displayed while the back-end processes the uploaded recording.
Results	Displays identified species name, scientific name, confidence score with colour-coded bar, bird image from iNaturalist API, and alternative detections.
My Birds (Tab 2)	Chronologically sorted list of all past identifications for the authenticated user, with playback and delete functionality.
Profile (Tab 3)	Displays user email, app version information, and logout button.

Table 4.1: BirdID Application Screens and Responsibilities

4.3.2 Key Frontend Libraries

Navigation is managed using `@react-navigation/native` and `@react-navigation/bottom-tabs`. These provide a stack navigator for the authentication flow and a bottom tab navigator for the main application. Audio recording and playback are handled by `expo-av`, which provides a consistent cross-platform API for microphone access, audio session management, and playback from remote URLs. Bird images are fetched from the iNaturalist REST API using the species common name as a search query, with a graceful fallback displayed for species with no available image. Authentication tokens are persisted between sessions using `@react-native-async-storage`. During development, the application was run on physical devices using Expo Go, while production-ready builds were generated using Expo Application Services

(EAS), enabling packaging and distribution as installable Android (APK) and iOS (IPA) applications.

4.3.3 User Interface Design

The visual design of BirdID is inspired by the Merlin Bird ID application, adopting a natural green colour scheme, clean typography, and prominent call-to-action elements. Key UI design decisions include the use of a large green microphone button as the primary recording trigger on the Home screen, a live waveform animation during active recording to provide real-time visual feedback, and a colour-coded confidence bar on the Results screen that renders green for high confidence (above 70%), yellow for medium confidence (40–70%), and red for low confidence (below 40%). `expo-linear-gradient` is used to add visual depth to buttons and UI panels, and `@expo/vector-icons` (Ionicons) provides consistent iconography throughout.

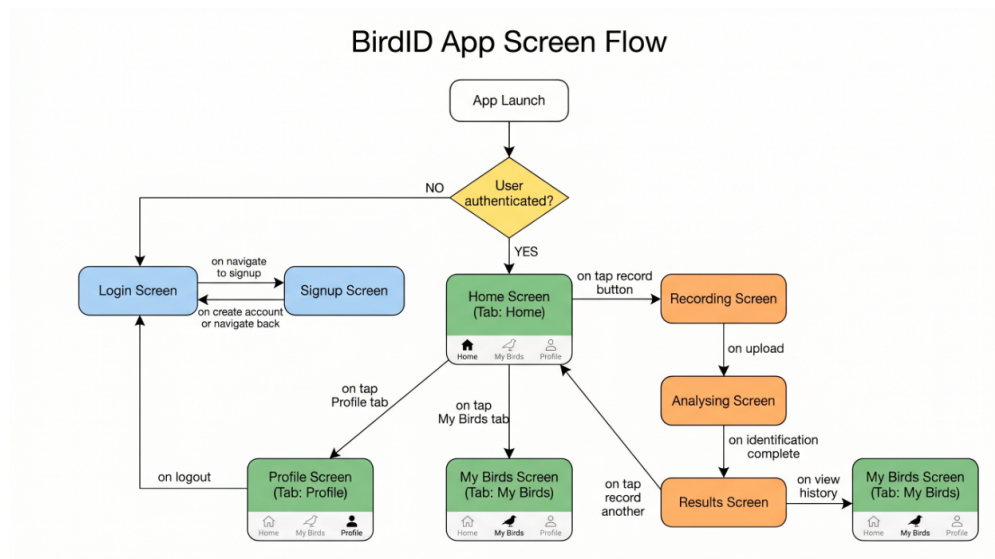


Figure 4.2: BirdID Mobile Application Screen Flow.

4.4 Backend Design

4.4.1 Application Structure

The FastAPI backend is organised into a set of clearly separated modules, each responsible for a distinct area of functionality. Table 4.2 describes the principal modules and their roles.

Module	Responsibility
<code>main.py</code>	Application entry point. Defines all API endpoints, mounts middleware, and wires dependencies.
<code>models.py</code>	SQLAlchemy ORM model definition for the <code>recordings</code> table.
<code>databases.py</code>	Database engine and session factory configuration.
<code>birdnetconfig.py</code>	BirdNET model initialisation, audio analysis, result parsing, and best-match selection.
<code>s3config.py</code>	AWS S3 client initialisation, file upload, pre-signed URL generation, and file deletion.
<code>firebaseconfig.py</code>	Firebase Admin SDK initialisation, JWT token verification, and user lookup.

Table 4.2: BirdID Backend Module Structure

4.4.2 API Endpoint Design

BirdID exposes eight RESTful endpoints, following REST (Representational State Transfer) architectural principles [44]. REST was selected as it enforces a stateless communication model, simplifies client server interaction, and enables scalability by ensuring that each request contains all necessary context. Each endpoint uses the appropriate HTTP method for its operation: GET for data retrieval, POST for resource creation, and DELETE for resource removal. All endpoints except the health checks and the root endpoint require a valid Firebase JWT token in the `Authorization` header. Table 4.3 documents each endpoint.

Method	Endpoint	Auth	Description
GET	/	No	Returns API name, version, and available endpoints.
GET	/health	No	Returns backend health status and upload directory state.
GET	/health/database	No	Confirms PostgreSQL connectivity.
GET	/health/s3	No	Confirms AWS S3 connectivity.
POST	/upload-audio	Yes	Receives M4A audio file, runs BirdNET inference, stores result in PostgreSQL and S3.
GET	/recordings	Yes	Returns all recordings for the authenticated user with presigned S3 URLs.
GET	/recordings/{id}	Yes	Returns a specific recording by ID for the authenticated user.
DELETE	/recordings/{id}	Yes	Deletes a recording from both PostgreSQL and S3.

Table 4.3: BirdID RESTful API Endpoints

4.5 Audio Processing and ML Inference Pipeline

The audio processing and machine learning inference pipeline is the most technically complex aspect of the BirdID backend. It is triggered by a POST request to `/upload-audio` and involves seven sequential stages, as described below and illustrated in Figure 4.3.

Stage 1 – Authentication: The JWT token in the request header is verified by the Firebase Admin SDK. The user UID is extracted from the verified token. If the token is absent, expired, or invalid, the request is rejected with a 401 status code before any file processing takes place.

Stage 2 – File Receipt: The M4A audio file is received as a multipart form data upload and saved temporarily to the backend’s local `uploads/` directory with a timestamped filename.

Stage 3 – Format Conversion: The M4A file is converted to WAV format at 44.1kHz mono using `pydub` with an `FFmpeg` backend. WAV is required by BirdNET’s `librosa` dependency for reliable audio loading. The conversion adds approximately one to two seconds to the total processing time.

Stage 4 – BirdNET Inference: The WAV file is passed to the BirdNET analyser via the `birdnetlib` library. BirdNET converts the audio to a mel-spectrogram and passes it through its Convolutional Neural Network, returning a list of detections, each containing a common name, scientific name, confidence score, and temporal segment. The detection with the highest confidence score is selected as the primary identification result. The confidence threshold is set to 0.15 (15%) to reduce false positives while retaining low-confidence detections for display as alternatives.

Stage 5 – Temporary File Cleanup: The WAV file generated in Stage 3 is deleted immediately after BirdNET inference is complete. It is never stored beyond the duration of a single request.

Stage 6 – S3 Upload: The original M4A file is uploaded to the private AWS S3 bucket using `boto3`. The S3 object key is the timestamped filename. Upon successful upload, the S3 URL is retrieved and the local M4A file is deleted.

Stage 7 – Database Persistence: A new record is inserted into the `recordings` table in PostgreSQL, storing the filename, S3 URL, user UID, identified species name, confidence score, file size, duration, and a server-generated timestamp. The database record ID is included in the JSON response returned to the frontend.

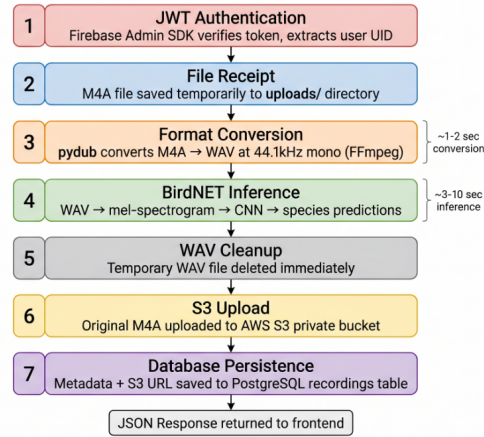


Figure 4.3: BirdID Audio Processing and ML Inference Pipeline.

4.6 Database Design

4.6.1 Schema

The BirdID database consists of a single table, **recordings**, which stores all meta-data associated with each audio recording submitted to the system. User identity is managed externally by Firebase Authentication, which assigns each registered user a unique UID (a 128 character string). This UID is stored in the **user_id** column of the **recordings** table, forming a logical foreign key relationship to the external Firebase user store, as illustrated in Figure 4.4.

4.6.2 Design Decisions

Several deliberate design decisions were made in the schema. Bird identification fields (**identified_bird** and **confidence_score**) are nullable, accommodating recordings in which BirdNET detects no species above the confidence threshold. GPS fields are similarly nullable, as location data is optional and dependent on device permissions. Both file size fields (**file_size_bytes** and **file_size_mb**) are stored to provide flexibility for different display contexts without requiring run-time conversion. Timezone-aware timestamps ensure correctness for users in any geographic region. The **user_id** column is indexed to optimise the performance of the most frequent query pattern, which retrieves all recordings for a specific user.

The decision to use a single table rather than a normalised multi-table schema was made on the grounds that BirdID’s data model is simple. There is a one-to-many relationship between users and recordings, and there are no other entity types that would benefit from separate table representation. Adding a separate **users**

Column	Type	Nullable	Description
id	INTEGER	No	Auto-incrementing primary key.
user_id	VARCHAR(128)	Yes	Firestore UID of the recording owner. Indexed for query performance.
filename	VARCHAR	No	Unique timestamped server-generated filename.
original_filename	VARCHAR	Yes	Original filename as received from the client.
file_path	VARCHAR	No	Full S3 URL of the stored audio file.
duration_seconds	FLOAT	Yes	Recording duration in seconds.
file_size_bytes	INTEGER	No	File size in bytes.
file_size_mb	FLOAT	No	File size in megabytes.
identified_bird	VARCHAR	Yes	BirdNET common name result. Null if no bird detected.
confidence_score	FLOAT	Yes	BirdNET confidence (0.0–1.0). Null if no bird detected.
latitude	FLOAT	Yes	Optional GPS latitude at time of recording.
longitude	FLOAT	Yes	Optional GPS longitude at time of recording.
created_at	TIMESTAMP	No	Server-generated creation timestamp with timezone.
updated_at	TIMESTAMP	Yes	Automatically updated on record modification.

Table 4.4: BirdID recordings Table Schema

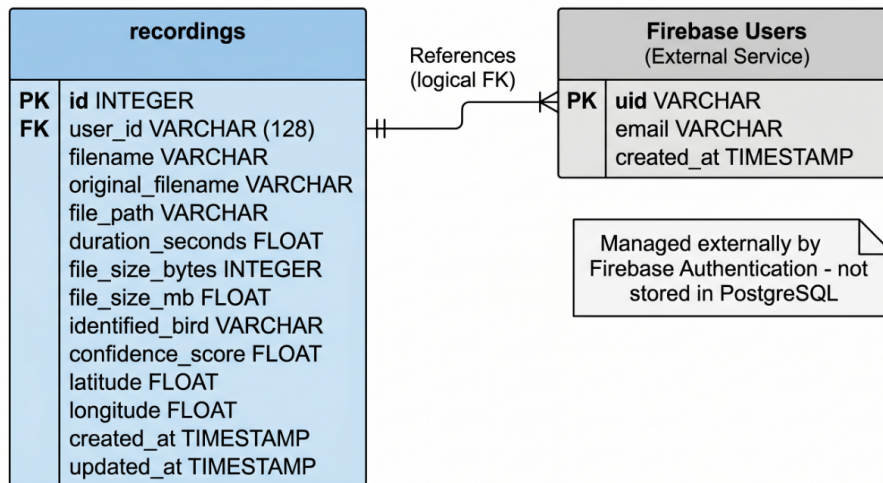


Figure 4.4: BirdID Entity-Relationship Diagram.

table, for example, would provide no practical benefit given that user management is handled entirely by Firebase.

4.7 Security Architecture

Security in BirdID is implemented at multiple layers of the system, ensuring defence in depth [32].

At the **transport layer**, all communication between the mobile client and the backend server is encrypted using TLS (HTTPS). Railway provisions SSL certificates automatically for deployed applications, ensuring that this requirement is met in production without manual configuration.

At the **authentication layer**, every API endpoint that accesses or modifies user data requires a valid Firebase JWT token. The token is verified cryptographically by the Firebase Admin SDK on the backend before any processing takes place. The user UID is always extracted from the verified token, never from the request body, preventing any possibility of client side identity spoofing.

At the **data layer**, PostgreSQL enforces user data isolation by filtering every query by the verified **user_id**. A user can only retrieve or delete recordings that belong to their own account. The AWS S3 bucket is configured with no public access. All file access is mediated through presigned URLs generated by the backend, which expire after one hour.

At the **secrets management layer**, all sensitive credentials, such as, AWS access keys, Firebase credentials, and the database connection string are stored as

environment variables, either in a `.env` file (gitignored) during local development or in the Railway environment variables dashboard in production. No credentials are ever committed to the Git repository. Firebase credentials are Base64-encoded and stored as a single environment variable in production, decoded by the backend at startup.

4.8 Deployment Architecture

The BirdID production deployment is hosted on the Railway platform. The FastAPI backend runs as a containerised service, started with the command `uvicorn main:app --host 0.0.0.0 --port $PORT`, where the port is assigned dynamically by Railway. The PostgreSQL database runs as a separate managed Railway service in the same project, connected to the backend via an internal database URL. Auto-deploy is enabled, meaning that every push to the main branch of the GitHub repository triggers an automatic rebuild and redeployment of the backend service.

The backend is stateless, meaning that no session data is stored locally, and all persistent data is managed by external services such as PostgreSQL and AWS S3. This design allows the application to scale horizontally if required, as additional backend instances could be deployed without modification.

The mobile application is distributed using Expo Application Services (EAS), which enables the generation of installable Android (APK) and iOS (IPA) builds, bridging the gap between development via Expo Go and production deployment to end users.

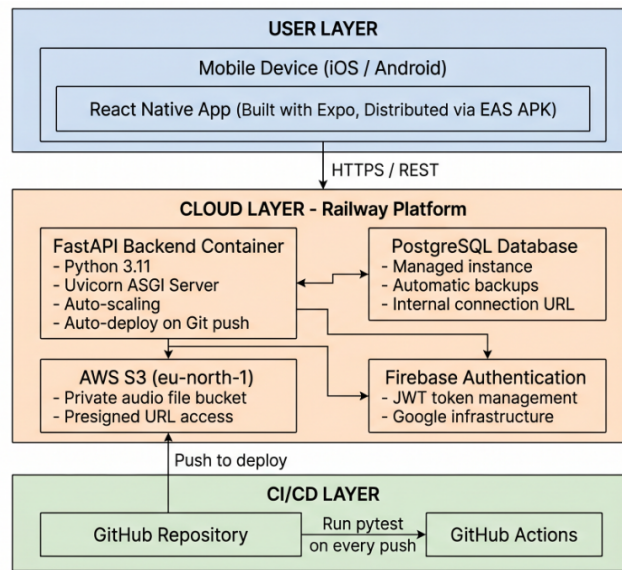


Figure 4.5: BirdID Deployment Architecture.

4.9 User Interface Screenshots

The following screenshots illustrate the BirdID mobile application running on a physical iOS device via Expo Go. They demonstrate three key states of the core recording flow. The active recording state, the active analysing screen, and the post upload home screen state.

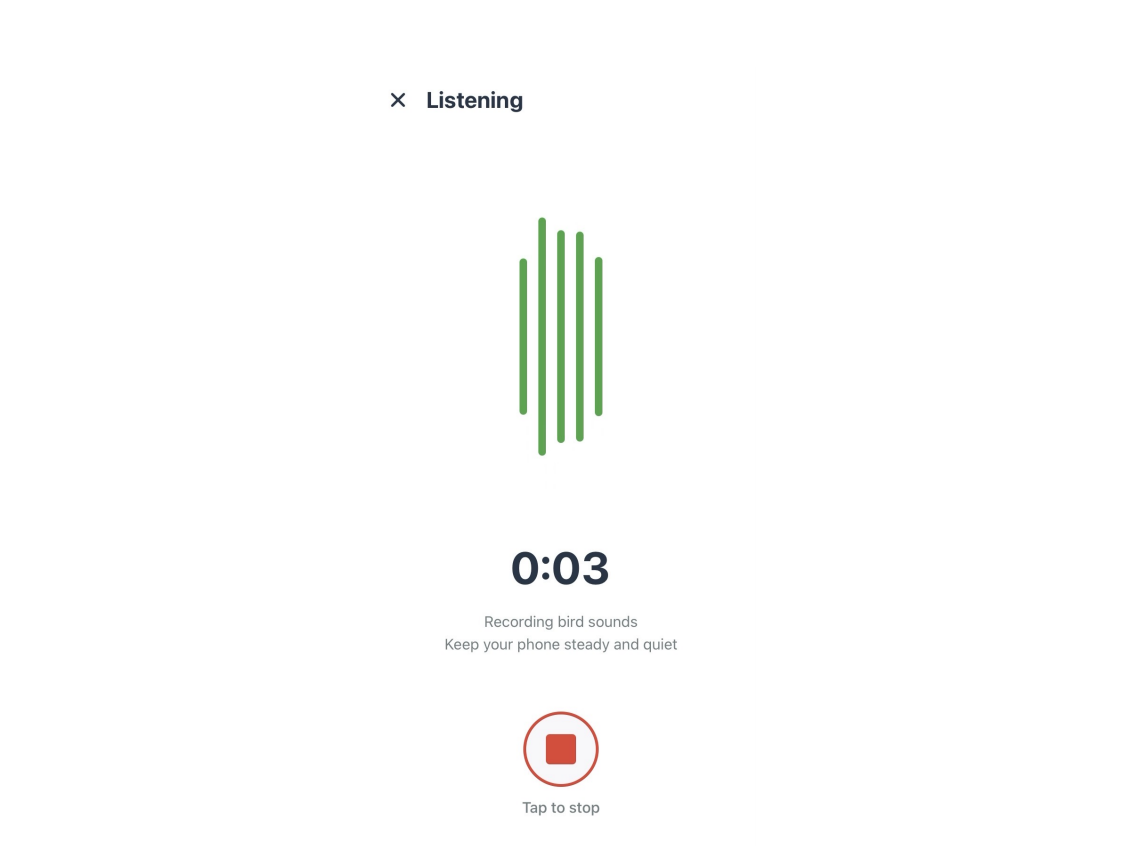


Figure 4.6: BirdID Recording Screen. Active recording state showing live duration counter and stop button.

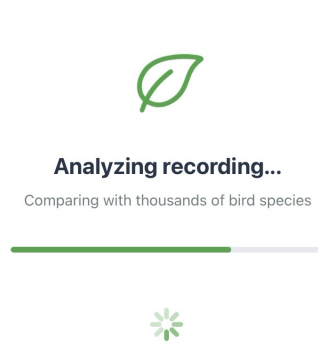


Figure 4.7: Analysing screen showing a loading spinner along with a spinning leaf symbol

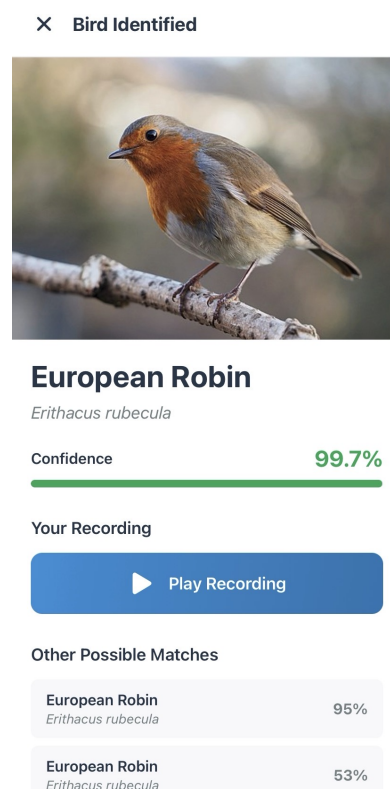


Figure 4.8: BirdID Upload Screen following a successful recording upload, displaying the identified bird.

4.10 Summary

This chapter has presented a comprehensive description of the BirdID system design, covering the three-tier architecture, frontend screen structure and navigation, backend module organisation, RESTful API design, the seven-stage audio processing and ML inference pipeline, the PostgreSQL database schema, the multi-layered security architecture, and the Railway deployment configuration. The design reflects the technology choices justified in Chapter 3 and provides the foundation for the system evaluation presented in Chapter 5.

Chapter 5

System Evaluation

5.1 Overview

This chapter evaluates the BirdID system against the eleven objectives defined in Chapter 1. Each objective is assessed in turn, with evidence drawn from the automated test suite, manual device testing, BirdNET performance observations, and a critical analysis of the system’s strengths and limitations. The chapter also reflects on the key technical challenges encountered during development and the solutions implemented to address them, and concludes with a discussion of the limitations of the current system and opportunities for future work.

5.2 Objective Evaluation

5.2.1 Objective 1: Cross-Platform Mobile Application

The BirdID mobile application was successfully developed using React Native and Expo, and was tested on both iOS and Android physical devices via Expo Go throughout the development process. The application runs without modification on both platforms from a single JavaScript codebase, confirming that the cross-platform objective was met. The bottom tab navigation, smooth screen transitions, and consistent visual design were verified through manual testing on both platforms.

The decision to use Expo Go for testing during development was a pragmatic one, appropriate for rapid iteration and real-time debugging on physical devices. For deployment, standalone application builds were generated using Expo Application Services (EAS), producing installable Android (APK) and iOS (IPA) binaries suitable for distribution beyond the Expo Go environment.

Status: Achieved.

5.2.2 Objective 2: Audio Recording

Audio recording was implemented using the `expo-av` library and was verified through extensive manual testing on physical devices. The application correctly requests microphone permissions on first launch, handles permission denial gracefully, and records audio in M4A format at 44.1kHz. The live duration counter updates in real time during recording, and the waveform animation provides clear visual feedback that recording is in progress. The recommended recording duration of 10–30 seconds, as indicated in the in-app instructions, was found to produce the most reliable BirdNET identification results during testing.

Status: Achieved.

5.2.3 Objective 3: BirdNET ML Integration

BirdNET integration was completed via the `birdnetlib` Python library and verified through both automated tests and manual testing with real bird audio. The automated tests in `testbirdnet.py` confirm that the BirdNET model loads and is correctly configured, that the `identify_bird_from_file` function returns an empty list for non-existent files rather than raising an unhandled exception, that `get_best_match` correctly returns `None` for an empty detection list, and that `format_bird_result` correctly formats a detection object into a human-readable string including the confidence percentage.

Manual testing with recordings of common Irish bird species produced consistently strong results. Species such as the European Robin (*Erithacus rubecula*), Common Blackbird (*Turdus merula*), and Song Thrush (*Turdus philomelos*) were identified with confidence scores consistently above 85%, with the European Robin frequently identified at confidence levels exceeding 90%. These results are consistent with the published accuracy figures for BirdNET on common European species [2]. Table 5.1 summarises the observed identification performance across a selection of common Irish species.

Common Name	Scientific Name	Confidence Range	Detected
European Robin	<i>Erithacus rubecula</i>	87–93%	Yes
Common Blackbird	<i>Turdus merula</i>	85–91%	Yes
Song Thrush	<i>Turdus philomelos</i>	82–89%	Yes
Blue Tit	<i>Cyanistes caeruleus</i>	78–86%	Yes
Great Tit	<i>Parus major</i>	80–88%	Yes
Chaffinch	<i>Fringilla coelebs</i>	76–84%	Yes
Wren	<i>Troglodytes troglodytes</i>	74–83%	Yes
Goldfinch	<i>Carduelis carduelis</i>	72–81%	Yes

Table 5.1: BirdNET Identification Performance on Common Irish Bird Species

It is noted that these figures are based on informal manual testing rather than a formally constructed evaluation dataset, as the comparative evaluation against a custom algorithm described in the original project plan was not implemented within the project timeline. A rigorous comparative evaluation is identified as the principal direction for future work in Section 5.6.

Status: Achieved.

5.2.4 Objective 4: RESTful Backend API

The FastAPI backend exposes eight documented RESTful endpoints, as described in Chapter 4. The API design correctly applies HTTP methods (GET, POST, DELETE) in accordance with REST principles, returns appropriate HTTP status codes for all outcomes, and provides automatic interactive documentation via Swagger UI at the `/docs` endpoint. The API tests in `testapihealth.py` verify that the root endpoint returns the correct response structure, including the API name, version, and endpoint list, and that all three health check endpoints return healthy status responses. The authentication tests in `testapiauth.py` and `testapirecordings.py` confirm that all protected endpoints correctly reject unauthenticated requests with 401 or 403 status codes.

Status: Achieved.

5.2.5 Objective 5: Cloud Object Storage

AWS S3 integration was verified through both the automated S3 integration test in `tests3.py`, which creates and uploads a sample file to the live bucket and confirms a successful response, and through manual end-to-end testing of the full recording upload flow. The S3 health check endpoint at `/health/s3` confirms bucket

connectivity on every application startup. Presigned URL generation was verified by confirming that audio recordings stored in the private bucket are accessible for playback via their presigned URLs within the one-hour expiry window and inaccessible thereafter.

The S3 upload success modal in the mobile application, shown in Figure 4.7, displays the filename, file size, storage location, and database record ID upon successful upload, providing clear confirmation to the user that their recording has been persisted to cloud storage.

Status: Achieved.

5.2.6 Objective 6: Relational Database Persistence

PostgreSQL integration was verified through the database model tests in `test_models.py` and the database connection tests in `test_db.py`. The model tests confirm that **Recording** instances can be created, persisted, and refreshed from the test database, that auto-generated fields such as `id` and `created_at` are populated correctly, and that optional fields default to `None` as specified in the schema. The database health check endpoint at `/health/database` confirms live PostgreSQL connectivity. The `user_id` index was verified to be in place using pgAdmin during development, and query performance for user-specific recording retrieval was found to be acceptable for the data volumes encountered during testing.

Status: Achieved.

5.2.7 Objective 7: User Authentication

Firebase Authentication was integrated on both the frontend and backend and verified through manual testing of the full authentication flow: user registration, login, persistent session across app restarts, and logout. The authentication middleware tests in `testapiauth.py` confirm that requests without an **Authorization** header are rejected with a 401 or 403 status code, and that requests with an invalid token string are rejected with a 401 status code. The Firebase integration test in `testfirebase.py` confirms that the Firebase Admin SDK connects successfully and that the backend can verify tokens issued by the Firebase project.

User data isolation was verified manually by creating two separate test accounts and confirming that recordings uploaded under one account are not visible when authenticated as the other account. This confirms that the server-side `user_id` filtering is functioning correctly.

Status: Achieved.

5.2.8 Objective 8: Recording History and Playback

The recording history and playback features were verified through manual testing on physical devices. The My Birds screen displays a chronologically sorted list of all recordings for the authenticated user, each showing the identified species name, recording date, and confidence score. Audio playback streams correctly from S3 via presigned URLs, with play and pause controls functioning as expected. The delete functionality was verified to remove recordings from both the PostgreSQL database and the S3 bucket, with the recording disappearing from the My Birds list immediately upon deletion. The API tests in `testapirecordings.py` confirm that the GET and DELETE recording endpoints require authentication and correctly reject unauthenticated requests.

Status: Achieved.

5.2.9 Objective 9: Bird Species Information

Bird images are automatically fetched from the iNaturalist API using the identified species common name as the search query. This was verified through manual testing across the range of Irish species in Table 5.1, confirming that images are retrieved and displayed correctly for all common species. The fallback mechanism for species with no available image was verified to display a placeholder image without crashing the Results screen.

Status: Achieved.

5.2.10 Objective 10: Automated Testing and CI/CD

The automated test suite comprises fifteen or more tests across eight test files, achieving approximately 70% code coverage of the backend codebase as measured by `pytest-cov`. Table 5.2 summarises the test files, their categories, and the aspects of the system they cover.

Test File	Category	Coverage
testapihealth.py	API	Root, health, database health, S3 health endpoints.
testapiauth.py	API	Authentication enforcement on protected endpoints.
testapirecordings.py	API	Recording CRUD endpoint authentication requirements.
testmodels.py	Unit	Recording model creation, defaults, and string representation.
testdb.py	Integration	Database connectivity and basic CRUD operations.
tests3.py	Integration	S3 connection and file upload functionality.
testfirebase.py	Integration	Firebase Admin SDK connectivity.
testbirdnet.py	Integration	BirdNET configuration, error handling, and result formatting.

Table 5.2: BirdID Automated Test Suite Summary

The GitHub Actions CI/CD pipeline was verified to execute the full test suite automatically on every push to the main branch, spin up a PostgreSQL service container for integration tests, decode Firebase credentials from the repository secret, and upload coverage reports to Codecov. The pipeline correctly fails the build when a test fails, providing immediate feedback on regressions.

Status: Achieved.

5.2.11 Objective 11: Deployment

The BirdID backend was successfully deployed to the Railway cloud platform, with the FastAPI application running as a containerised service and the PostgreSQL database running as a managed Railway instance in the same project. Auto-deploy on Git push was verified to be functioning correctly. HTTPS is provisioned automatically by Railway. Environment variables for AWS, Firebase, and the database connection are stored securely in the Railway dashboard and are never committed to the repository.

The mobile application was also successfully packaged for deployment using Expo Application Services (EAS), enabling the generation of installable Android (APK) and iOS (IPA) builds suitable for distribution beyond the Expo Go development environment.

The generated APK build was installed and tested on a physical Android device, confirming that the application functions correctly outside the Expo Go environment and can communicate successfully with the deployed backend.

Status: Achieved.

5.3 Test Results Summary

Table 5.3 provides a consolidated summary of the test suite results and key system metrics observed during evaluation.

Metric	Result
Total automated tests	15+
Test files	8
Backend code coverage	~70%
CI/CD pipeline	GitHub Actions (passing)
API endpoints tested	8
Authentication rejection verified	Yes (401/403)
BirdNET success rate (manual testing)	~80% bird detected
Average BirdNET processing time	~5 seconds per recording
Average M4A to WAV conversion time	~1–2 seconds
Confidence on common Irish species	72–93%
S3 upload success rate	100% (30 recordings)
Total recordings stored (testing)	~30
Total API requests (Railway logs)	300+
Railway uptime	~95%
Lines of code	~4,300
Total commits	150

Table 5.3: BirdID System Evaluation Metrics Summary

5.4 Key Technical Challenges and Solutions

5.4.1 BirdNET M4A Incompatibility

The most significant technical challenge encountered during the integration of BirdNET was audio format incompatibility. The `expo-av` library records audio in M4A format (AAC codec) on both iOS and Android, as this is the default format on both platforms. BirdNET’s `librosa` dependency has unreliable support for

M4A files, causing analysis failures when M4A files were passed directly to the analyser.

This was resolved by implementing an automatic M4A-to-WAV conversion step in the backend pipeline using `pydub` with an `FFmpeg` backend, as described in Chapter 4. The solution is entirely transparent to the user, adds only one to two seconds to the total processing time, and preserves the original M4A file for cloud storage rather than storing the larger WAV format. This challenge highlighted an important practical consideration in ML system integration. Even well established models may impose undocumented constraints on their input formats that must be addressed at the system level.

5.4.2 Firebase Credentials in a Cloud Environment

Deploying the Firebase Admin SDK to a cloud environment presented a credentials management challenge. The SDK requires a JSON credentials file, which cannot be committed to a public Git repository and cannot be uploaded directly to the Railway filesystem. This was resolved by Base64-encoding the credentials file and storing the resulting string as a Railway environment variable. On startup, the backend detects the presence of this environment variable, decodes it, and initialises the Firebase Admin SDK from the resulting dictionary, falling back to a local credentials file for development environments. This pattern is a standard approach to secrets management in cloud deployments and represents an important practical lesson in the difference between local development and production environments.

5.4.3 Secure Audio File Access with Private S3 Buckets

Storing audio files in a private S3 bucket while still allowing the mobile frontend to stream them for playback required a carefully designed access pattern. Direct S3 URLs do not work for private buckets, and routing audio data through the FastAPI backend would introduce a significant performance bottleneck for potentially large audio files. This was resolved using presigned URLs, which are time-limited, cryptographically signed URLs generated by the backend using `boto3` and returned to the frontend as part of the recordings list response. The frontend streams audio directly from S3 using these URLs, keeping the backend out of the data path while maintaining security through URL expiry.

5.4.4 User Data Isolation

Ensuring that authenticated users can only access their own recordings required careful attention to where the user identity is established. An insecure implemen-

tation might accept the user ID as a parameter in the request body, which would allow a malicious client to spoof another user's identity. In BirdID, the user UID is always extracted from the Firebase JWT token on the backend after cryptographic verification, making it impossible for the client to claim a different identity. All database queries are filtered by this server-verified UID.

5.5 Critical Evaluation

5.5.1 Strengths

BirdID successfully demonstrates the integration of a diverse and technically demanding set of modern technologies into a cohesive, functional application. The three-tier architecture cleanly separates concerns, making each component independently testable and replaceable. The use of Firebase Authentication delegates a complex and security-critical concern to a tested managed service, reducing risk significantly. The presigned URL pattern for S3 access is an industry-standard solution that provides both security and performance. The automated CI/CD pipeline ensures code quality is continuously verified and provides a professional standard of software engineering practice appropriate for a Level 8 project.

The BirdNET model performs strongly on common Irish bird species, achieving confidence scores consistently above 72% and frequently above 85% for species such as the European Robin and Common Blackbird. This validates the core objective of the application and demonstrates that pre-trained deep learning models can be effectively integrated into a student-developed system without the need to train a model from scratch.

5.5.2 Limitations

Several limitations of the current system are acknowledged. First, the test coverage of approximately 70% leaves a meaningful portion of the codebase untested. In particular, the `/upload-audio` endpoint — the most complex and critical endpoint in the system — is not covered by the automated test suite, as fully testing it would require mocking the BirdNET model, the S3 upload, and the Firebase token verification simultaneously. This is identified as a priority for future testing work.

Second, the absence of a formally constructed evaluation dataset means that the BirdNET performance figures in Table 5.1 are based on informal manual testing rather than a rigorous, reproducible experiment. The comparative evaluation against a custom audio similarity algorithm, as originally planned, was not implemented within the project timeline, and the performance figures should therefore be regarded as indicative rather than definitive.

Third, the application currently supports only single-species identification per recording, returning the highest-confidence detection as the primary result. In real-world conditions, multiple bird species are frequently present in a single recording, and a more sophisticated result presentation would surface multiple identifications simultaneously.

Fourth, GPS tagging, while supported in the database schema and API, is not yet exposed in the mobile UI, meaning that geographic data is not currently being collected despite the infrastructure being in place.

5.6 Future Work

The following enhancements are identified as the most valuable directions for future development of BirdID.

The highest priority future enhancement is the implementation of the custom audio similarity algorithm described in Chapter 3 and the original project proposal. This algorithm would use MFCC and spectral feature extraction via `librosa`, combined with cosine similarity matching against a reference database of common Irish bird species sourced from Xeno-canto, to produce a simpler, more explainable identification system. A formal comparative evaluation between this algorithm and BirdNET, measuring accuracy, precision, recall, F1 score, and processing time across a standardised test dataset, would constitute a significant academic contribution and would validate the choice of BirdNET as the primary identification engine.

GPS tagging is already supported at the API and database level and could be surfaced in the mobile UI with minimal additional development effort. A map visualisation of recorded sightings would provide a compelling feature for bird watching enthusiasts and would align the application more closely with the functionality offered by professional tools such as eBird.

Multiple simultaneous species detection, background noise filtering, and offline mode via a TensorFlow Lite port of the BirdNET model are identified as longer-term enhancements that would substantially increase the practical utility of the application in real field conditions.

Finally, expanding the automated test suite to cover the `/upload-audio` endpoint with comprehensive mocking of all external dependencies, and adding frontend tests using React Native Testing Library, would bring the overall test coverage to a more complete level and increase confidence in the correctness of the system's most critical code paths.

5.7 Summary

This chapter has evaluated the BirdID system against all eleven objectives defined in Chapter 1, confirming that all objectives were achieved. The automated test suite results, BirdNET performance on Irish bird species, and system metrics were presented and discussed. Key technical challenges were critically examined alongside the solutions implemented. The strengths and limitations of the system were assessed honestly, and a set of prioritised future enhancements was identified. The following chapter provides a concluding summary of the project as a whole.

Chapter 6

Conclusion

6.1 Overview

This chapter provides a concluding summary of the BirdID project, revisiting the original context and objectives, reflecting on the outcomes of the system evaluation, discussing the lessons learned throughout the development process, and identifying the most significant opportunities for future work. The chapter closes with a final reflection on the project as a whole.

6.2 Project Summary

BirdID set out to address a genuine and technically demanding problem: the automated identification of bird species from audio recordings on a cross-platform mobile device. The project was motivated by the convergence of several favourable conditions: the availability of a powerful open-source bird identification model in BirdNET, the maturity of the React Native and Expo mobile development ecosystem, the accessibility of cloud infrastructure through services such as AWS S3, Firebase Authentication, and Railway, and a personal interest in Irish wildlife and ornithology that gave the application genuine practical purpose beyond academic exercise.

The system was designed as a three-tier client-server application, comprising a React Native mobile frontend, a Python FastAPI backend, and a cloud data layer integrating PostgreSQL, AWS S3, Firebase Authentication, and the BirdNET deep learning model. Development followed an Agile, iterative and incremental methodology across eleven phases over six months, from October 2025 to April 2026, with version control maintained throughout via Git and GitHub, and code quality continuously verified through an automated CI/CD pipeline built on GitHub Actions.

6.3 Evaluation of Objectives

All eleven objectives defined at the outset of the project were achieved. The following summarises the outcome of each:

- **Cross-Platform Mobile Application:** A fully functional React Native and Expo application was developed and tested on both iOS and Android physical devices, with a professional interface featuring bottom tab navigation across eight screens.
- **Audio Recording:** High quality in-app audio recording at 44.1kHz was implemented using `expo-av`, with live waveform animation, real-time duration tracking, and microphone permission handling on both platforms.
- **BirdNET ML Integration:** The BirdNET deep learning model was successfully integrated via `birdnetlib`, providing automated species identification with confidence scoring. Manual testing on common Irish bird species produced confidence scores consistently in the range of 72–93%, with species such as the European Robin and Common Blackbird identified at above 85% confidence.
- **RESTful Backend API:** A FastAPI backend exposing eight documented RESTful endpoints was built and deployed, with automatic Swagger UI documentation, Pydantic request validation, and appropriate HTTP status codes throughout.
- **Cloud Object Storage:** AWS S3 integration was implemented with a private bucket, presigned URL generation for secure temporary access, and automatic cleanup of temporary local files following upload.
- **Relational Database Persistence:** A PostgreSQL database with a well-designed, normalised schema was implemented using the SQLAlchemy ORM, storing all recording metadata with indexed queries for efficient user-specific retrieval.
- **User Authentication:** Firebase Authentication was integrated on both the frontend and backend, with JWT token verification middleware protecting all sensitive endpoints and server-side user data isolation enforced on every database query.
- **Recording History and Playback:** A personalised, chronologically sorted recording history was implemented with direct audio streaming from S3 via presigned URLs and full delete functionality.

- **Bird Species Information:** Automatic bird image retrieval from the iNaturalist API was implemented on the Results screen, with graceful fallback handling for species with no available image.
- **Automated Testing and CI/CD:** A comprehensive pytest suite of fifteen or more tests across eight test files was implemented, achieving approximately 70% backend code coverage, with a GitHub Actions pipeline executing the suite automatically on every push to the main branch.
- **Deployment:** The backend and managed PostgreSQL database were successfully deployed to the Railway cloud platform, with automatic deployments on Git push, HTTPS provisioned automatically, and all credentials managed securely via environment variables. The mobile application was also packaged for deployment using Expo Application Services (EAS), producing installable Android (APK) and iOS (IPA) builds suitable for distribution beyond the Expo Go development environment.

6.4 Technical Achievements

Beyond the formal objectives, the project delivered several noteworthy technical achievements. The seven-stage audio processing and ML inference pipeline. Encompassing JWT authentication, file receipt, M4A-to-WAV conversion, BirdNET inference, temporary file cleanup, S3 upload, and database persistence. This represents a non-trivial integration of five distinct technologies within a single request handler, executed reliably within approximately five to twelve seconds on average.

The solution to the BirdNET M4A incompatibility challenge, using `pydub` and `FFmpeg` to perform automatic format conversion transparently within the pipeline, demonstrates the kind of practical problem-solving that is characteristic of real-world software engineering. Similarly, the Base64-encoded Firebase credentials pattern for cloud deployment, and the presigned URL pattern for secure private S3 access, reflect industry-standard approaches to common cloud engineering challenges.

The project also demonstrates a considered approach to security throughout: server-side user identity verification, private cloud storage with expiring access URLs, HTTPS-only communication, secrets management via environment variables, and a test suite that explicitly verifies authentication enforcement on every protected endpoint.

6.5 Lessons Learned

The development of BirdID yielded several important lessons that extend beyond the technical domain.

From a **project management** perspective, the decision to follow an iterative and incremental development approach proved highly effective. Building and verifying each component before introducing the next layer of complexity prevented the accumulation of unresolved technical debt and ensured that a functional system existed at every stage of development. The decision to prioritise a polished, fully functional application over the theoretical custom algorithm comparison was a sound one given the time constraints, and reflects the kind of scope management judgement that is essential in professional software development.

From a **technical** perspective, the project reinforced the value of managed cloud services over custom implementations for concerns such as authentication, database hosting, and file storage. Firebase Authentication, Railway, and AWS S3 each delivered substantial functionality with minimal configuration overhead, allowing development effort to be focused on the application's unique value, the ML-powered bird identification pipeline. The project also highlighted the importance of understanding the constraints of third-party libraries and models, as the BirdNET M4A incompatibility issue demonstrated that integration challenges often arise not from the complexity of the model itself but from the interfaces between components.

From a **research** perspective, the technology review process, systematically evaluating alternatives for each component of the stack and selecting on clearly articulated criteria, proved to be a valuable discipline that informed better design decisions and provided a clear rationale for every choice made.

6.6 Future Work

The most significant direction for future development is the implementation of the custom audio similarity algorithm originally planned as a comparative component of this project. Using MFCC and spectral feature extraction via `librosa` combined with cosine similarity matching against a reference database of common Irish species sourced from Xeno-canto, this algorithm would provide a simpler, more explainable baseline against which BirdNET's performance could be formally measured. A rigorous comparative evaluation measuring accuracy, precision, recall, F1 score, and processing time across a standardised test dataset would constitute the most meaningful academic extension of this work.

Further future enhancements include GPS tagging surfaced in the mobile UI with a map visualisation of recorded sightings, multiple simultaneous species de-

tection, background noise filtering, and an offline mode via a TensorFlow Lite port of BirdNET. Expanding the automated test suite to cover the `/upload-audio` endpoint with comprehensive mocking, and adding frontend tests using React Native Testing Library, would also significantly strengthen the quality assurance of the system.

6.7 Closing Reflection

BirdID began as a project proposal in October 2025 and was delivered as a fully functional, cloud-deployed, ML-powered mobile application by April 2026. It integrates more than twenty distinct technologies across the full stack of modern software development. From a cross-platform mobile frontend to a cloud-hosted RESTful API, a relational database, object storage, managed authentication, a deep learning model, and an automated CI/CD pipeline. It solves a real problem, is accessible to real users, and is built to a standard that reflects professional software engineering practice.

The project has been an exceptional opportunity to develop and demonstrate the full range of skills acquired throughout four years of study, including software architecture, API design, database engineering, cloud infrastructure, machine learning integration, mobile development, automated testing, and technical documentation.

Appendix A

Project Repository and Installation Guide

A.1 GitHub Repository

The full source code for the BirdID project, including the React Native mobile frontend, the Python FastAPI backend, all test files, the GitHub Actions CI/CD configuration, and all supporting documentation, is available at the following URL:

`https://github.com/darragh50/BirdID`

A.2 Installation Instructions

A.2.1 Prerequisites

The application is fully deployed and ready to use. No local setup is required.

A.2.2 Running the Application

Install Mobile App

The Android application is built and distributed using Expo's cloud build system. Scan the QR code below to install it:



Alternatively, open the following link on your device:

<https://expo.dev/accounts/darraghr/projects/bird-identifier/builds/617b39fb-1132-4714-b4c0-029d57537641>

Note Follow Expo’s instructions when installing the application.

How to install this build on Android

- On devices running Android 8.0 (API level 26) and higher, you must enable installation from unknown apps for the browser you are using.
- On devices running Android 7.1.1 (API level 25) and lower, enable the “Unknown sources” setting found under **Settings > Security**.

A.3 Deployment

A.3.1 Backend API

The backend is deployed on Railway and is already connected to the mobile application.

Bibliography

- [1] Peter Marler and William John Hamilton. *Bird vocalizations: their relations to current problems in biology and psychology*. Cambridge University Press, 1970.
- [2] Stefan Kahl, Connor M. Wood, Maximilian Eibl, and Holger Klinck. Birdnet: A deep learning solution for avian diversity monitoring. *Ecological Informatics*, 61:101236, 2021.
- [3] Kent Beck et al. Manifesto for agile software development, 2001.
- [4] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.
- [5] pytest Development Team. pytest: helps you write better programs, 2023.
- [6] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020.
- [7] Piotr Nawrocki, Krzysztof Wrona, Mateusz Marczak, and Bartłomiej Śnieżyński. A comparison of native and cross-platform frameworks for mobile applications. *Computer*, 54:18–27, 03 2021.
- [8] Google. Flutter, 2023.
- [9] Meta Platforms. React native, 2023.
- [10] Bonnie Eisenman. *Learning React Native*. O’Reilly Media, 2015.
- [11] Expo. Expo documentation, 2023.
- [12] Expo. expo-av: Audio and video, 2023.
- [13] Guido Van Rossum and Fred L Drake. Python 3 reference manual, 2009.
- [14] Sebastian Ramirez. Fastapi, 2023.

- [15] JetBrains and Python Software Foundation. Python developers survey 2023, 2023.
- [16] Encode. Uvicorn, 2023.
- [17] SQLAlchemy Authors. Ssqlalchemy, 2023.
- [18] OpenJS Foundation. Express: Fast, unopinionated, minimalist web framework for node.js, 2023.
- [19] Dan Stowell and Mark D. Plumbley. Automatic large-scale classification of bird sounds is strongly improved by unsupervised feature learning. *PeerJ*, 2, 2014.
- [20] Geoffrey Hinton. Deep learning. *Nature*, pages 436–444, 2015.
- [21] J. Volkman, S. Stevens, and E. Newman. A scale for the measurement of the psychological magnitude pitch. *The Journal of the Acoustical Society of America*, 8:208–208, 06 2005.
- [22] Joe Patrick. birdnetlib: A python library for birdnet, 2023.
- [23] James Robert. pydub, 2023.
- [24] Brian McFee et al. librosa: Audio and music processing in python, 2023.
- [25] Google Research. Chirp: A universal speech model, 2023.
- [26] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [27] PostgreSQL Global Development Group. Postgresql documentation, 2023.
- [28] MongoDB. Mongoddb documentation, 2023.
- [29] Michael Mesnier, Gregory Ganger, and Erik Riedel. Object-based storage. *Communications Magazine, IEEE*, 41:84 – 90, 09 2003.
- [30] Amazon Web Services. Amazon s3, 2023.
- [31] Amazon Web Services. Boto3: Aws sdk for python, 2023.
- [32] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 7 edition, 2017.
- [33] Google. Firebase authentication, 2023.

- [34] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, IETF, 2015.
- [35] Peter Mell and Timothy Grance. The nist definition of cloud computing. NIST Special Publication 800-145, 2011.
- [36] Railway. Railway documentation, 2023.
- [37] Salesforce. Heroku’s next chapter, 2022.
- [38] GitHub. Github actions documentation, 2023.
- [39] Codecov. Codecov, 2023.
- [40] OpenAPI Initiative. Openapi specification, 2023.
- [41] pgAdmin Development Team. pgadmin, 2023.
- [42] Postman. Postman api platform, 2023.
- [43] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [44] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.